Configuración + PostgreSQL

Primero verifica la base de datos. Después arranca la API y el móvil.

Preparación local y consulta impresa - no sustituye la comprensión del código.

01 / Antes de quitar internet

Usa Node.js 22 y las dependencias instaladas. Abre una vez Expo en el dispositivo. Prueba todo con internet desconectado. Conserva node_modules en tu equipo; no se entrega.

```
node --version
npm --version
psql --version

# Preparación previa, con internet:
cd backend
npm install
cd ../frontend
npm install
```

02 / Crea y comprueba la BD

Abre psql desde la terminal con el primer comando. Después escribe las instrucciones SQL y los comandos con barra inversa dentro de psql.

```
psql -h localhost -U postgres -d postgres

CREATE DATABASE control_databridge;
\c control_databridge
SELECT current_database();
\dt
-- Después de arrancar la API:
SELECT * FROM items;
\q
```

03 / Conecta NestJS

En TypeOrmModule.forRoot, el usuario, la contraseña y la BD deben coincidir con PostgreSQL. synchronize solo se usa aquí como configuración de laboratorio.

```
type: "postgres",
host: "localhost", port: 5432,
username: "postgres",
password: "TU_CLAVE_LOCAL",
database: "control_databridge",
autoLoadEntities: true,
synchronize: true,
```

04 / Arranca en este orden

PostgreSQL debe estar iniciado como servicio. Mantén dos terminales abiertas: una para NestJS y otra para Expo. No instales paquetes durante el control.

```
cd backend
npm run start:dev

# Segunda terminal:
cd frontend
npx --no-install expo start --offline
```

05 / Cada dispositivo tiene su host

psql y NestJS: localhost.
Expo Go en móvil físico: IP del ordenador, misma red local.
Emulador Android de Android Studio: 10.0.2.2 para acceder a la API del ordenador.
localhost en el móvil apunta al móvil. Sin internet no significa sin red local.

```
// En el frontend, ejemplo LAN:
const API = "http://192.168.1.20:3000";

// En main.ts del backend:
app.enableCors();
await app.listen(3000, "0.0.0.0");
```

06 / Diagnóstico rápido

psql no se reconoce: instalación o PATH.
password authentication failed: credenciales.
database does not exist: crea la BD correcta.
ECONNREFUSED: servicio, host o puerto.
Network request failed: API, IP, firewall o misma LAN.
Puerto 3000 ocupado: detén el proceso anterior.
Prueba la API desde el ordenador antes de revisar React Native.

Backend: del HTTP al dato

Controller recibe; Service decide; Repository consulta; PostgreSQL conserva.

Preparación local y consulta impresa - no sustituye la comprensión del código.

01 / Las piezas del dominio

En el módulo de la funcionalidad registra la Entity con TypeOrmModule.forFeature([Item]). Incluye Controller y Service; importa ese módulo desde AppModule.

```
@Entity("items")
export class Item {
  @PrimaryGeneratedColumn() id: number;
  @Column() titulo: string;
  @Column({ default: false })
  completado: boolean;
}
```

02 / Repository en el Service

Los decoradores proceden de @nestjs/common, @nestjs/typeorm y typeorm, según la pieza. El DTO describe entradas; no sustituye a la Entity.

```
constructor(
  @InjectRepository(Item)
  private readonly repo: Repository<Item>
) {}

// Crear y persistir:
const item = this.repo.create(dto);
return this.repo.save(item);
```

03 / Buscar y controlar el 404

El id recibido por HTTP debe convertirse a número. Si no existe, devuelve un 404 con NotFoundException. No confíes en un objeto undefined.

```
const item = await this.repo.findOneBy({id});
if (!item) {
  throw new NotFoundException();
}
return item;
```

04 / Un verbo, una intención

Controller delega en Service; evita escribir SQL o lógica de persistencia en el Controller. Un DTO TypeScript por sí solo no valida los datos en ejecución.

```
@Get()           // colección + @Query
@Get(":id")      // detalle + @Param
@Post()          // crear + @Body
@Patch(":id")    // actualizar parcialmente
@Delete(":id")   // eliminar

@Param("id", ParseIntPipe) id: number
```

05 / Cambiar sin borrar lo demás

PATCH recibe campos opcionales. Primero localiza el registro. merge aplica cambios al objeto; save persiste. Para borrar puedes cargar y después usar remove.

```
const item = await this.findOne(id);
this.repo.merge(item, dto);
return this.repo.save(item);

// En remove(id):
const item = await this.findOne(id);
await this.repo.remove(item);
return { eliminado: true };
```

06 / Filtros opcionales

Añade a where únicamente los parámetros recibidos. Ambos campos del mismo objeto se combinan con AND. Adapta las propiedades a la Entity.

```
const where: FindOptionsWhere<Item> = {};
if (ciudad) where.ciudad = ciudad;
if (categoria) where.categoria = categoria;
return this.repo.find({ where });

// Importa FindOptionsWhere de typeorm.
// Item debe declarar ciudad/categoria.
```

Frontend: pedir y sincronizar

API_URL correcta + método HTTP + JSON coherente + actualización de estado.

Preparación local y consulta impresa - no sustituye la comprensión del código.

01 / Lee una colección

Mantén la colección en estado. Llama a cargar al abrir la pantalla; los errores deben mostrarse al usuario. FlatList debe utilizar una clave estable.

```
type Item = { id: number; titulo: string;
  completado: boolean };
const [items, setItems] =
  useState<Item[]>([]);

async function cargar() {
  const r = await fetch(API + "/items");
  if (!r.ok) throw new Error("HTTP "+r.status);
  setItems(await r.json());
}
useEffect(() => {
  cargar().catch(() => setError("Sin datos"));
}, []);
```

02 / Envía el formulario

Los valores deben proceder de TextInput controlados. Convierte los números con Number y comprueba el resultado antes de enviarlos.

```
const r = await fetch(API + "/items", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({ titulo })
});
if (!r.ok) throw new Error("No se guardó");
await cargar();
```

03 / PATCH y DELETE

En cada petición comprueba r.ok. En PATCH envía solo lo que cambia. Después vuelve a cargar la colección o actualiza el estado de forma coherente.

```
await fetch(API + "/items/" + id, {
  method: "PATCH",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({completado: true})
});
// DELETE: misma URL, method: "DELETE".
```

04 / Construye los filtros

La interfaz debe permitir elegir o escribir los filtros. Codifica los valores y omite los vacíos. Descarga el resultado filtrado desde la API.

```
const q = [];
if (ciudad) q.push(
  "ciudad=" + encodeURIComponent(ciudad));
if (categoria) q.push(
  "categoria=" + encodeURIComponent(categoria));
const url = API + "/items" +
  (q.length ? "?" + q.join("&") : "");
const r = await fetch(url);
```

05 / Prueba por capas

1. GET desde el ordenador.
2. POST y consulta en PostgreSQL.
3. PATCH de un solo campo.
4. GET con filtros combinados.
5. DELETE y ausencia en la BD.
6. Repite desde el móvil.
7. Reinicia NestJS: los datos deben seguir.

```
curl http://localhost:3000/items
curl --get http://localhost:3000/items \
  --data-urlencode "ciudad=Zaragoza"
```

06 / Antes de entregar

Backend y frontend en sus carpetas; README con comandos, configuración local y limitaciones reales. Guarda todos los archivos. No incluyas node_modules ni credenciales reales.

En el control, entrega por el medio local indicado por el profesor. GitHub, Forms y Classroom se usarán cuando se restablezca internet; no dependas de ellos durante los 60 minutos.

Regla de oro: que el README diga que funciona no demuestra que funcione.